

元智能平台

版本 1.08

杨国梁

目录

前言 **1**

第一章 引言 **2**

1.1 平台简介 2

1.2 联系 2

第二章 配置环境 **3**

2.1 检验是否安装 3

2.2 安装 jdk-11 3

2.2.1 下载 3

2.2.2 安装 4

2.2.3 配置环境变量 5

第三章 快速入门 **8**

3.1 运行平台 8

3.1.1 安装 yuan-common 8

3.1.2 运行 yuan-platform 9

3.1.3 打包 yuan-platform 10

3.2 运行 Maven 项目 10

3.3 运行 Python 11

3.4 Launch4 生成 EXE 12

3.4.1 Launch4j 简介 12

3.4.2 环境准备 12

3.4.3 详细打包步骤 12

3.5 小结 14

第四章 添加插件 **15**

4.1 快速使用插件 15

4.2 创建插件 16

4.3 编译和发布插件 16

第五章 视频检测 **18**
5.1 环境配置 18
5.1.1 环境 18
5.1.2 导出 yolov8s 模型 18
5.2 启动程序 19
5.2.1 打视频源 20
5.2.2 目标追踪 22
5.2.3 切换中英文标签 23
5.2.4 分类截图 23

前言

本文档是元智能平台（Yuan）的使用手册，对应版本 1.08。全书按章组织：第 2 章介绍环境配置，第 3 章介绍平台的快速入门与基本使用，第 4 章介绍如何添加自定义插件，第 5 章介绍基于 Yolov8s 的视频检测插件。

联系方式、版本与维护信息见第 1 章末尾的 “联系” 一节。

第一章　引言

Yuan 是专门为 Ai 研发的平台，由北京力控元通科技有限公司山东分公司支持开源，并维护到 1.0.6 版本。后续版本由作者杨国梁独立维护。

系统分为平台和算法库两部分，Yuan 平台是元智能系统中的平台部分，简称为 Yuan。Yuan 当前包括 yuan-common 和 yuan-platform 两部分。yuan-common 提供通用的接口和工具等内容，可以被扩展插件和 yuan-platform 使用。yuan-platform 是平台主体部分。

Yuan 是专门为 AI 算法研发、设计、运行、发布和应用而设计，当前支持 Java 和 Python 编辑和运行，只在 Windows 操作系统测试过。支持 Java 标准的 Maven 项目的编辑、编译、运行、打包、安装到仓库，添加自定义插件等基本的功能。支持 Python 的编辑、运行等功能。

Yuan 平台插件实现了 Java 调用 Yolov8s 实现视频检测功能。

1.1　平台简介

第 1 章引言部分。

第 2 章介绍了 Yuan 的环境配置。

第 3 章介绍了怎么快速入门使用 Yuan，并使用 Yuan 安装了 yuan-common 到仓库，运行了 yuan-platform 源文件。

第 4 章介绍了 Yuan 如何添加自定义插件。

第 5 章介绍了 Java 实现视频检测。

1.2　联系

系统技术支持，请联系杨国梁：Guoliang-Yang@hotmail.com

v1.06 以前版本业务合作，请联系尤从国（北京力控元通科技有限公司山东分公司）：youcongguo@126.com。v1.0.6 后，由杨国梁以独立身份维护。

本文最新修改时间为 2026 年 6 月 19 日。

第二章　配置环境

Yuan 当前测试了 Window 操作系统的使用。Yuan 使用了 apache-maven-3.9.0 和 jdk-11，发布文件中内置了 apache-maven-3.9.0。通过 2.1 节检验安装情况，运行 Java 时，通常，发布文件中没有提供内置的 jdk-11，则需要用户自己安装，并配置环境变量，具体可参见 2.2 节。

2.1　检验是否安装

Win + R 键输入 cmd，如图 2-1 所示。进入到 Windows 控制台，然后输入 `java -version`，出现`'java'` 不是内部或外部命令，也不是可运行的程序或批处理文件，说明未安装。

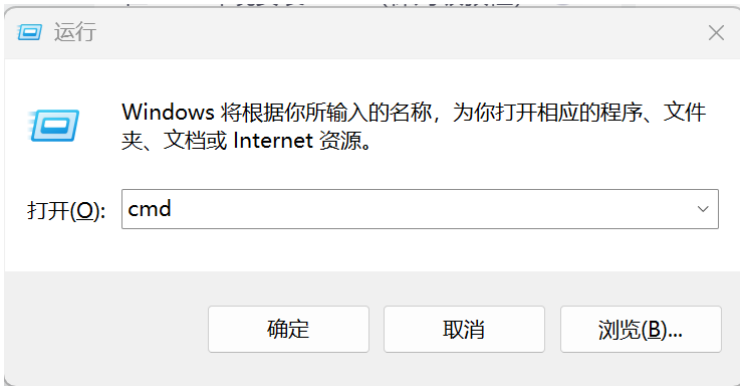


图 2-1 打开命令行

如果出现了类似图 2-10 的提示信息，说明安装成功，可以跳过 2.2 节，直接使用 Yuan。

2.2　安装 jdk-11

本节基于 jdk-11，演示下载安装和配置过程。如果已经安装可以跳过本节。

2.2.1　下载

官网地址 https://www.oracle.com/java/technologies/downloads/，借鉴图 2-2 寻找 jdk-11 的版本，并下载安装。需要自己注册一个 Oracle 账号，下载的时候需要登录。本文选择

的版本是 jdk-11.0.25_windows-x64_bin.exe。jdk-11.0.2 版本在编译 Yuan 时出现了英文代码注释乱码问题，建议使用 jdk-11.0.25。

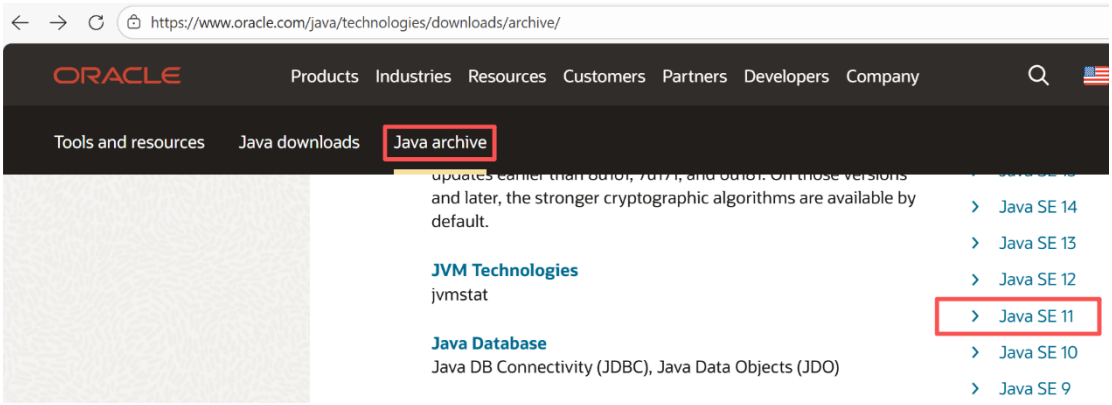


图 2-2 找到 jdk-11

2.2.2　安装

双击打开下载好的文件，出现图 2-3，点击下一步。



图 2-3 进入安装界面。

在图 2-4 中，选择合适的安装目录。



图 2-4 选择安装目录

2.2.3　配置环境变量

本文以 Win11 为例演示添加 jdk 环境变量。不同系统的环境变量设置方式，可通过网络查询具体方法。



图 2-5 电脑属性

右击“此电脑”选择属性，如图 2-5 所示，在弹出的如图 2-6 设置窗口中搜索“环境变量”，选择“编辑系统环境变量”。

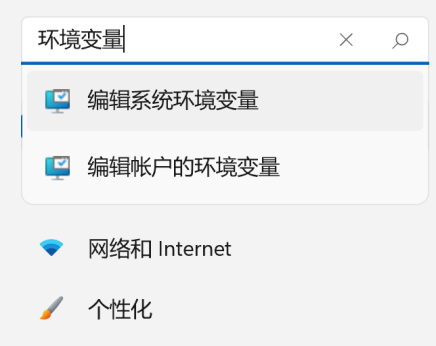


图 2-6 选择系统环境变量

在弹出的图 2-7 的“系统属性”中，点击环境变量按钮。



图 2-7 选择环境变量

2.2.3.1　添加 JAVA_HOME

在系统变量中点击新建，设置变量名为 JAVA_HOME，变量值填入刚刚安装的 JDK 的路径，图 2-8 中的“变量值”用自己电脑 jdk 的实际路径替代。

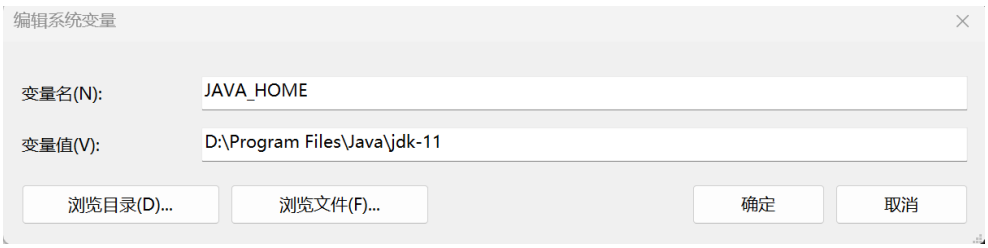


图 2-8 添加 JAVA_HOME

2.2.3.2　配置 Path

Path 变量中包含了系统可以执行的文件路径。当你在命令行中输入 java 或 javac 命令时，操作系统会根据 Path 环境变量的配置查找可执行文件。

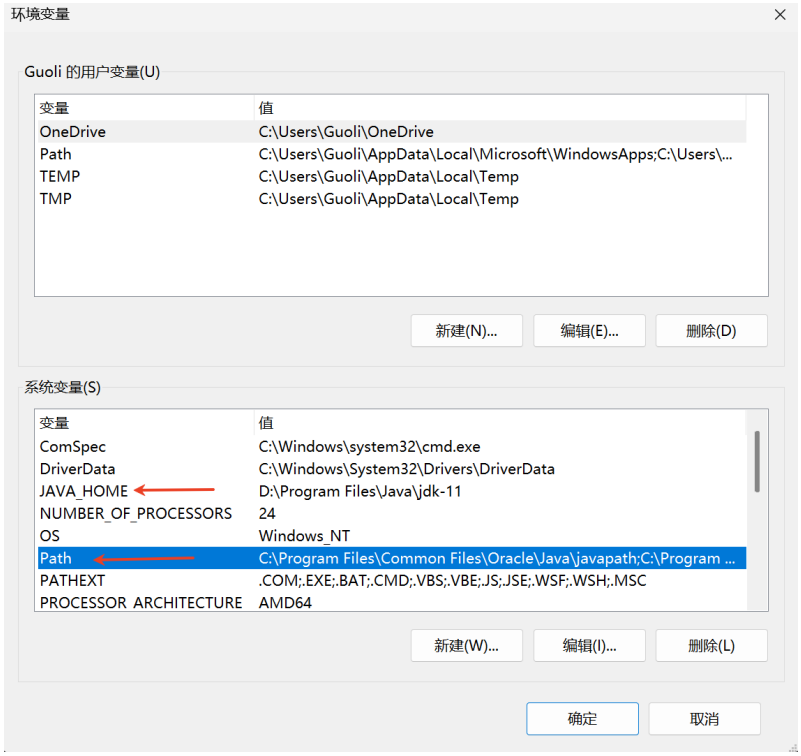


图 2-9 环境变量

在图 2-9“环境变量” 对话框中，找到并选中 “系统变量” 区域中的 Path 变量，然后点击 “编辑”，添加`%JAVA_HOME%\bin` 到 Path 中。

```
C:\Users\Guoli>java -version
java version "11.0.25" 2024-10-15 LTS
Java(TM) SE Runtime Environment 18.9 (build 11.
0.25+9-LTS-256)
Java HotSpot(TM) 64-Bit Server VM 18.9 (build 1
1.0.25+9-LTS-256, mixed mode)
```

图 2-10 安装 jdk-11 成功

按照 2.1 节检验安装，如果出现了类似图 2-10 的提示信息，说明安装成功，可以跳过 2.2 节，直接使用 Yuan。

第三章　快速入门

按第 2 章配置了平台运行环境后，在 https://github.com/YuanIdea/yuan 下载 Yuan 源码，双击后出现图 3-1 平台界面，不同的版本会略有差异。注意，不要直接使用 `yuan\yuan-platform\target\artifacts\yuan\yuan.exe`，因为使用这个目录下的可执行文件时，打包 yuan-platform 会因 `yuan-platform.jar` 正被 `yuan.exe` 占用而出错，需要先把 `yuan\yuan-platform\target\artifacts\yuan` 拷贝到其它目录后再打开。

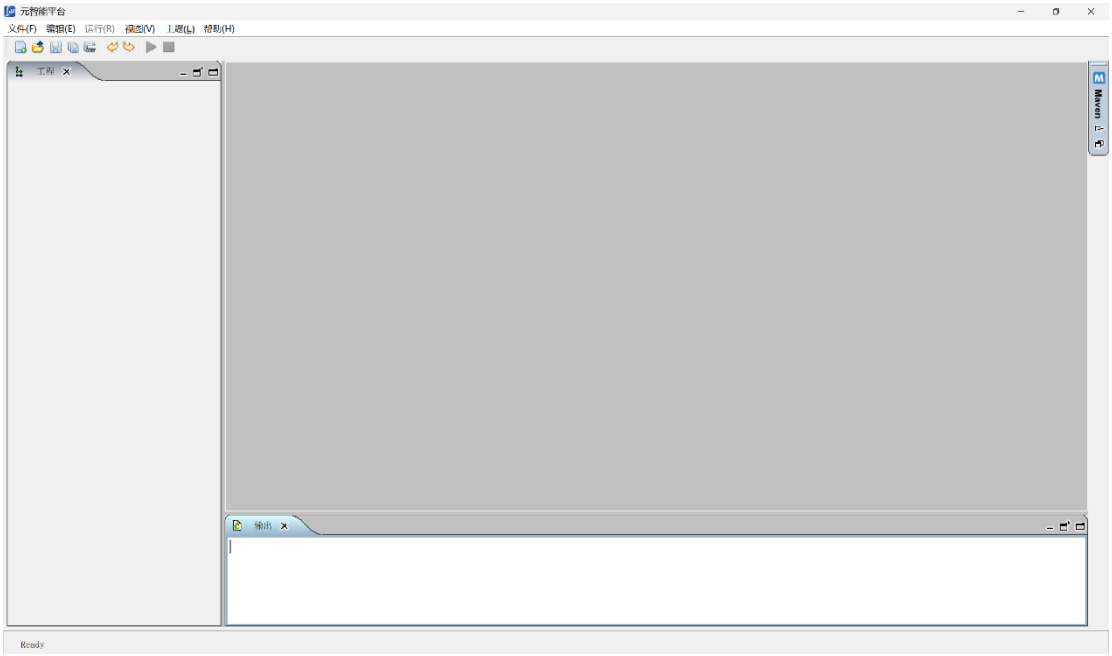


图 3-1 平台界面

3.1　运行平台

本节介绍在 Yuan 平台内编译，安装依赖和打包 Yuan 平台。

3.1.1　安装 yuan-common

选择平台中的 “文件”→“打开目录” 菜单栏。选择 yuan-common 的根目录。在右边的 maven 停靠窗口的 “生存期” 中，双击 “compile” 命令，运行效果如图 3-2 所示。在底部的输出停靠窗口中，会出现 “ compile succeeded!” 等编译信息。

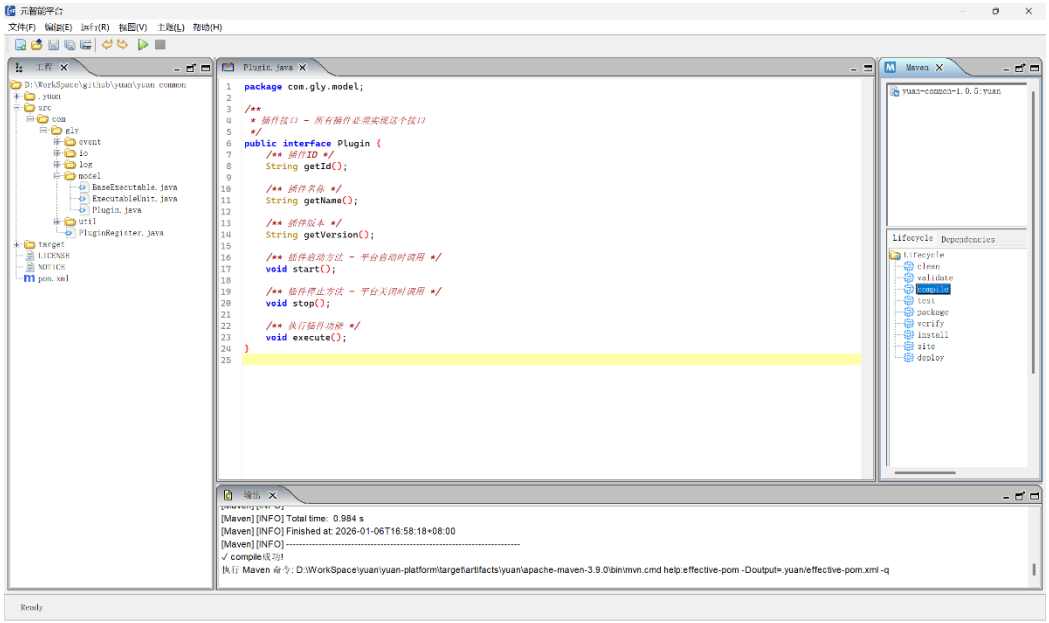


图 3-2 编译 yuan-common

点击 “install” 命令，将 yuan-common 安装到仓库，成功后输出面板如图 3-3 所示。

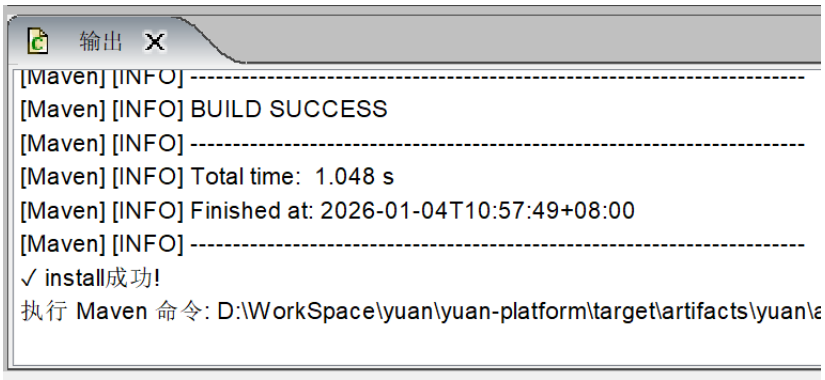


图 3-3 安装到仓库

3.1.2　运行 yuan-platform

安装完 yuan-platform 后，就可以运行 yuan-platform 了。同样，可以使用工具栏的 “打开目录” 按钮，打开 yuan-platform 的根目录。点击工具栏的绿色三角 “运行” 按钮，可以看到编译和运行 yuan-platform 源码后的效果为图 3-4。

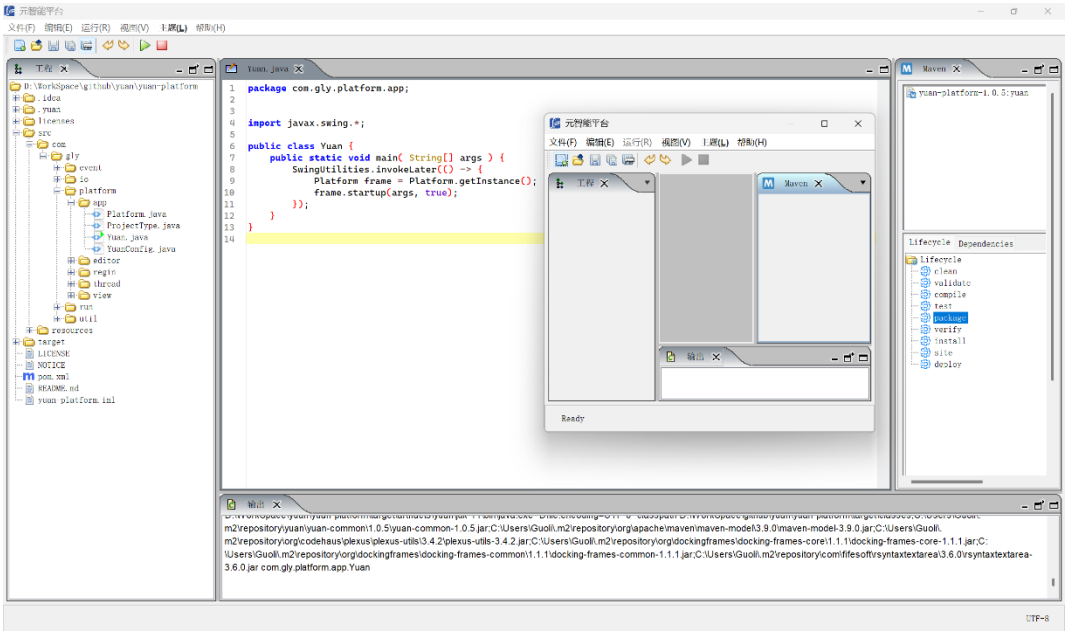


图 3-4 运行 yuan-platform

3.1.3　打包 yuan-platform

右边的 maven 停靠窗口的 Lifecycle 中，双击 “package” 命令后，将 yuan-platform 打包到了 yuan-platform/target/artifacts/yuan/yuan-platform-1.0.8.jar 中，在命令行中切换到 yuan-platform/target/artifacts/yuan 目录后使用 java -jar yuan-platform-1.0.8.jar 命令，同样可以打开图 3-1 所示界面。

3.2　运行 Maven 项目

Yuan 平台支持对标准类型 Maven 项目，以 yuan 平台使用的第 3 方 docking-frames 为例，在当前要运行的 “工程” 资源管理中添加配置文件.yuan/project.xml，见配置 3-1。

列表 3.1 配置 3-1 project.xml

```
1  <?xml version='1.0'?>
2  <project>
3  <type>maven</type>
4  <jdk>D:\Program Files\Java\jdk-1.8</jdk>
5  </project>
```

其中，type 指定了是 maven 类型的项目，jdk 指定了当前工程中使用的 jdk 存储路径，docking-frames 选择 jdk-1.8 版本进行编译，如果没有设置 jdk 标签，默认使用的是 JAVA_HOME 中指定的 JDK，见 2.2.3.1 节。

在右边的 Maven 停靠面板中选择 docking-frames-demo-chess，将需要的依赖添加到仓库后，点击运行按钮出现图 3-5。

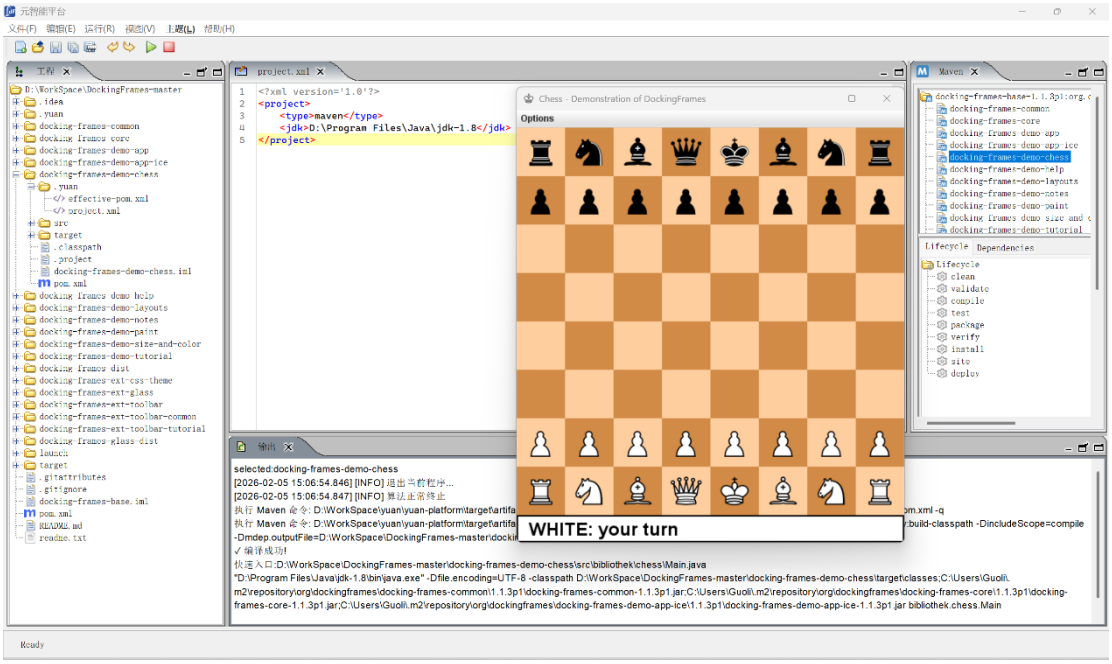


图 3-5 运行第 3 方开源 Java 工程。

注意，在运行工程前，需要将所有的本地依赖，使用 Maven 中的 install 暗装到仓库。通常，在使用 yuan 平台打开包含 pom.xml 的根目录时，平台会默认创建一个.yuan/project.xml 的配置文件，用户可以根据需要修改配置内容。

3.3　运行 Python

使用 Yuan 平台运行 Python 代码前，需要自行安装 python 环境，Yuan 只提供了编辑和运行 Python 的功能。安装 Python 成功后，在命令行中输入 python 命令，如果类似现出图 3-6 所示 Python 版本信息，则安装成功。

```
C:\Users\Guoli>python
Python 3.13.11 | packaged by Anaconda, Inc. | (main, Dec 10 2025, 21:21:58)
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

图 3-6 python 已安装效果

在 yuan 平台打开项目后，添加配置文件.yuan/project.xml，见配置 3-2。

列表 3.2 配置 3-2 project.xml

```
1  <?xml version='1.0'?>
2  <project>
3  <type>python</type>
4  <pythonHome>D:/ProgramData/anaconda3/envs/python310</pythonHome>
5  </project>
```

其中，type 指定了是 Python 类型的项目，pythonHome 指定了当前工程中使用的 python.exe 存储路径。

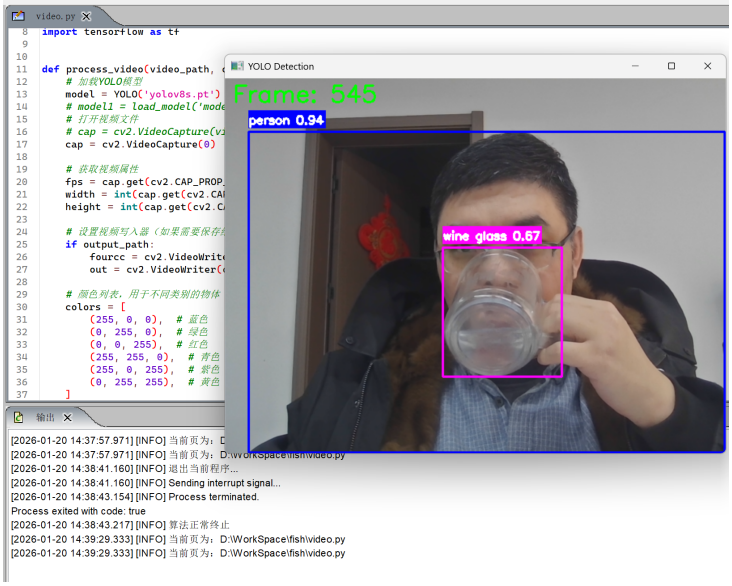


图 3-7 图像识别

图 3-7 演示了就是 Yuan 平台中，通过 Python 调用 yolov 模型，进行图像识别的过程。

3.4　Launch4 生成 EXE

本文使用了一款 Launch4j 工具将 Yun 的 JAR 包封装为 Windows 原生可执行文件.exe。如果不需要生成.exe，可以跳过本节。

3.4.1　Launch4j 简介

Launch4j 是一款开源工具，可将 Java 应用程序的 JAR 包封装为 Windows 原生可执行文件 (exe)，解决用户端无 JRE 环境时的运行问题。其优势在于支持自定义图标、JRE 版本绑定、启动参数配置等，且生成的 EXE 文件兼容性强。

下载路径为 [https://sourceforge.net/projects/launch4j/files/launch4j-3/3.50/]，选择 launch4j-3.50-win32.exe 进行下载。双击下载的 launch4j-3.50-win32.exe 自行安装。

3.4.2　环境准备

Java 环境：确保本地已安装 JDK/JRE，并配置好环境变量，见 2.2 节。

JAR 文件：需打包的 Java 程序需已编译为可执行 JAR，见 3.1.3 节。

3.4.3　详细打包步骤

启动安装好的 Launch4j，打开配置文件 yuan\yuan-platform\target\artifacts\yuan_jdk.xml，其中 Basic 选项卡如图 3-8 所示。

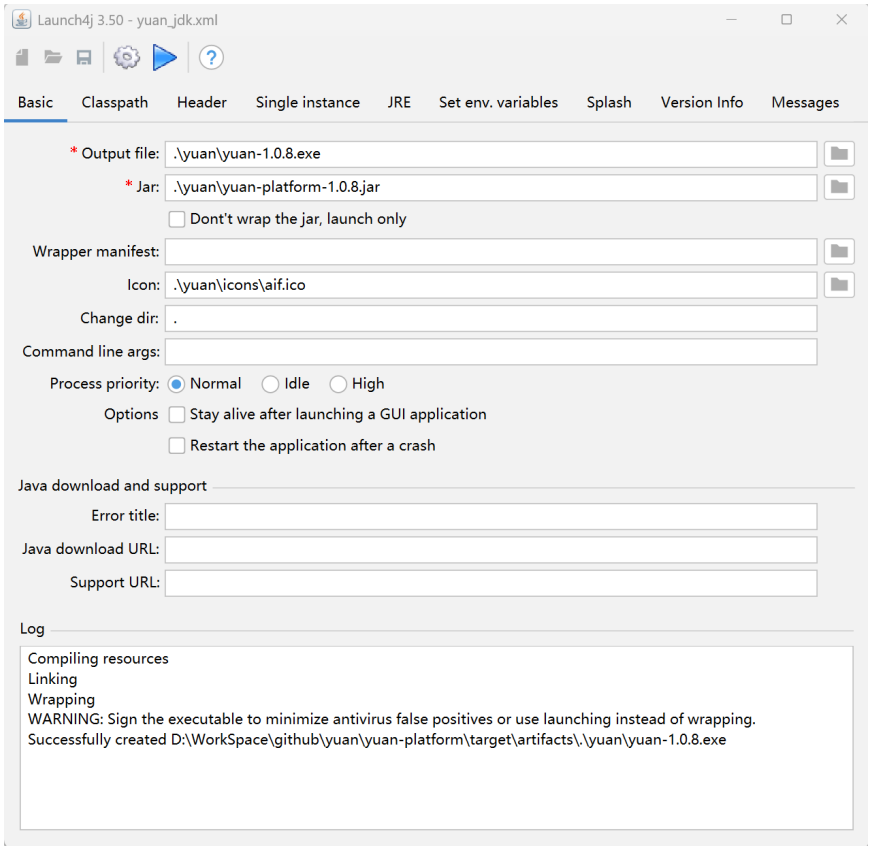


图 3-8 配置 yuan_jdk.xml

其中，Output file：设置生成的 EXE 路径；Jar：选择待打包的 JAR 文件路径；Icon（可选）：添加 EXE 图标（需.ico 格式）。

进入 JRE 选项卡，根据设置 JRE 的路径，如图 3-9 所示。

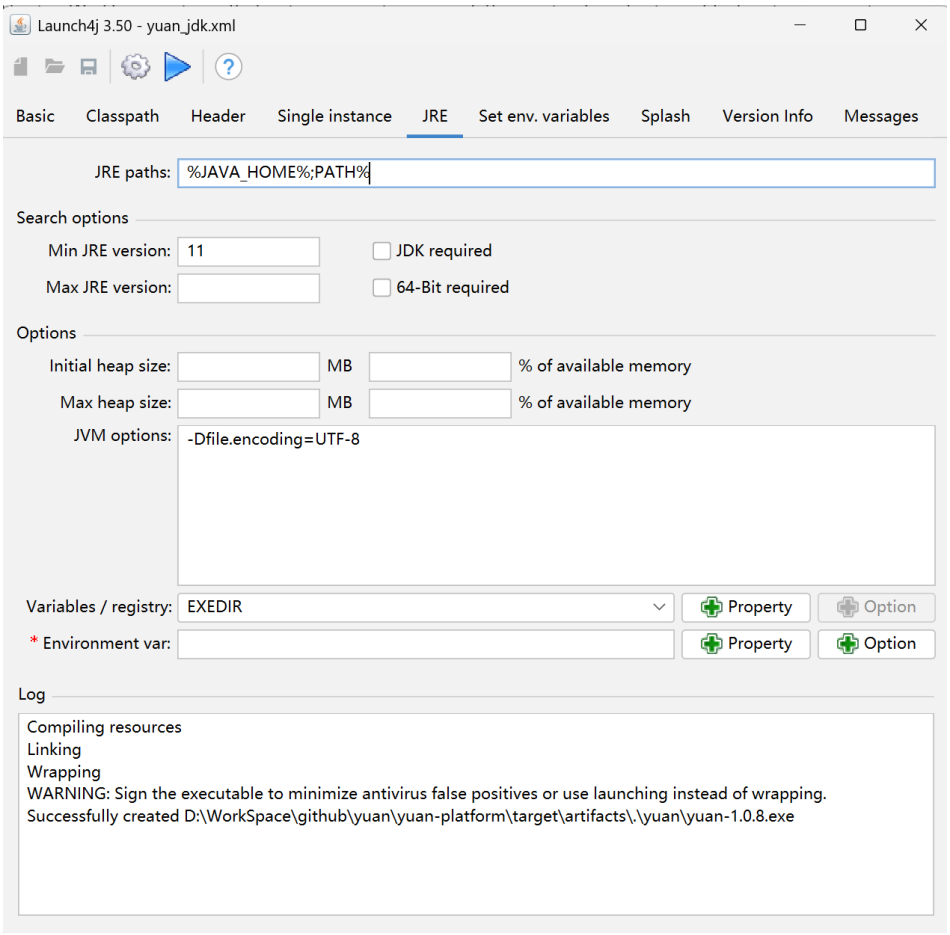


图 3-9 JRE 信息

Min JRE version：填写最低兼容的 JRE 版本。Bundled JRE path：指定相对路径。JVM Options：可以添加需要的参数，譬如-Dfile.encoding=UTF-8。

其它参数根据需要自行调整，配置完成后，点击 Build wrapper 按钮后，yuan\yuan-platform\target\artifacts\yuan 路径下可以看到生成的 yuan-1.0.8.exe。

3.5　小结

本节介绍了 Yuan 的使用，并在 Yuan 内编译安装了 yuan-common 到仓库。并在 Yuan 中编译运行了 yuan-platform，以及生成 yuan.exe 可执行文件。演示了 Yuan 的具体使用。当前 Yuan 只支持标准的 Maven 项目和简单的 Python 运行。

第四章　添加插件

在添加插件前，首先，参考第 2 章配置平台运行时必要的环境。然后，参考第 3 章的 3.1.1 节安装插件依赖的 yuan-common。

4.1　快速使用插件

Yuan 平台中已经自带了一个绘图插件可以直接使用,其存放目录为 `D:\WorkSpace\github\yuan\yuan-platform\target\artifacts\yuan\plugins\`。平台启动时，在“输出”面板中如果出现了“ Plugin loaded successfully:com.gly.Chart”，则说明插件可以直接使用。

Yuan 平台中也提供了一个默认的 yuan-chart 插件测试项目。使用元智能平台打开\yuan\yuan-demo\quick 项目，打开“chart.json”文件。点击工具栏中的三角形的运行按钮，运行效果如图 4-1 所示。

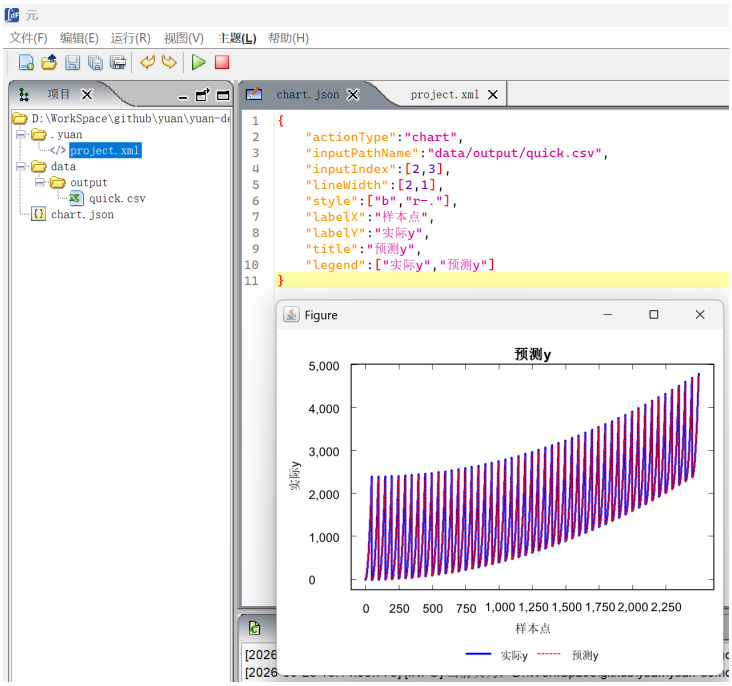


图 4-1 插件测试项目

在 yuan\yuan-demo\quick\.yuan\project.xml 目录下配置文件内容为配置 4-1，如果没有此配置文件，工具栏的“运行”按钮无法使用。

列表 4.1 配置 4-1 project.xml

```
1  <project>
2  <type>model</type>
3  </project>
```

4.2　创建插件

以 plot-chart 插件为例，yuan 平台的插件的 pom.xml 中只显示了必须包含 yuan-common 依赖，实现 yuan 平台规定的插件接口。配置 4-2 中，plot-chart 依赖的 yuan-common 版本是 1.0.8。

列表 4.2 配置 4-2 pom.xml

```
1  <dependency>
2  <groupId>yuan</groupId>
3  <artifactId>yuan-common</artifactId>
4  <version>1.0.8</version>
5  </dependency>
```

插件的源文件需要放在 src/com/gly 目录下，例如 plot-char 的入口类 Chart.java 的包名为 “package com.gly”。

插件的入口类必须实现 yuan-common 中定义的 ExecutableUnit 接口。例如，plot-chart 插件的入口类 Chart.java 继承了 yuan-common 中的抽象类 BaseExecutable，而 BaseExecutable 实现了 ExecutableUnit 接口。

此外，自定义的入口类中，如果使用 json 配置文件，可以自行封装解析逻辑。实现完成后，就可以通过 4.2 节介绍的方法添加自己实现的插件了。

4.3　编译和发布插件

使用 Yuan 打开 yuan\plugins\yuan-chart 项目，选择 Maven 面板中的 Package 命令，如图 4-2 所示，运行成功后在 yuan\plugins\yuan-chart\target 目录下就会生成一个 plot-chart-1.0.0.jar 文件。

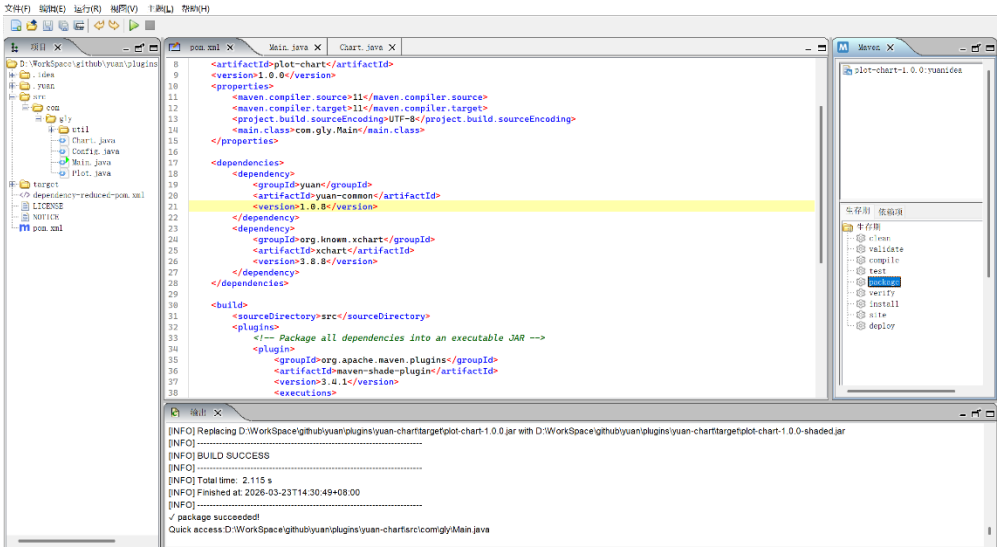


图 4-2 yuan-chart 生成 jar

yuan\yuan-platform\target\artifacts\yuan\plugins\plot-chart\lib 将生成的 plot-chart-1.0.0.jar 拷贝到 yuan\yuan-platform\target\artifacts\yuan\plugins\plot-chart\lib 目录后，重新启动 Yuan 平台，插件就添加完成。如果想将插件去掉，只需要将 plot-char 文件夹整体删除就可以了。

第五章　视频检测

本项目当前支持对 MP4 视频，笔记本自带摄像头，网络摄像头的目标追踪，以中文和英文方式显示标签。支持对视频中的目标图像，自动分类抠图。

在使用视频检测前，到 https://github.com/YuanIdea/yuan 下载源码，其中，`yuan\plugins\yuan-detection` 目录下为 Java 版的视频检测项目。

5.1　环境配置

本章介绍如何通过 DJL 在 java 中调用 yolov8 进行推理。Deep Java Library 是由亚马逊（Amazon）提供的一个深度学习工具包，能够让 java 开发者在 java 上调用目前主流的深度学习框架，像 pytorch、tensorflow、mxnet、paddlepaddle, 也包括 onnx 格式的模型。

5.1.1　环境

YuanIdea1.8 or Idea、pytorch-native-cpu-2.7.1、java11、Deep Java Library0.36.0、yolov8s、javacv1.5.11

5.1.2　导出 yolov8s 模型

本章节直接使用 yolov8s 的预训练模型，导出模型前，用户需要自行下载 yolov8s.pt 文件放在与导出代码 5-1 同目录下，模型导出脚本是代码 5-1。

导出成功后显示信息为下面所示，注意，本文版本 python3.13，torch-2.9.1。

```
1 Ultralytics 8.3.250 Python-3.13.11 torch-2.9.1+cu128 CPU (AMD Ryzen
    ↪ AI 9 HX 370 w/ Radeon 890M)
2 YOLOv8s summary (fused): 72 layers, 11,156,544 parameters, 0
    ↪ gradients, 28.6 GFLOPs
3 PyTorch: starting from 'yolov8s.pt' with input shape (1, 3, 640, 640)
    ↪ BCHW and output shape(s) (1, 84, 8400) (21.5 MB)
4 TorchScript: starting export with torch 2.9.1+cu128...
5 TorchScript: export success 2.1s, saved as 'yolov8s.torchscript'
    ↪ (43.0 MB)
6 Export complete (2.8s)
```

列表 5.1 代码 5-1 export.py

```python
from ultralytics import YOLO


def export_torchscript_model():
    model_path = "yolov8s.pt"
    input_size = 640

    model = YOLO(model_path)
    model.export(
        format="torchscript",
        imgsz=input_size,
        device="cpu",
        half=False,
        dynamic=False,
        simplify=True
    )


export_torchscript_model()
```

```
Results saved to D:\WorkSpace\fish
Predict: yolo predict task=detect model=yolov8s.torchscript imgsz=640
Validate: yolo val task=detect model=yolov8s.torchscript imgsz=640
    ↪ data=coco.yaml
Visualize: https://netron.app
Process finished with exit code 0
```

将自己导出的或者是上面网址中下载的 yolov8s.torchscript 文件，放在 yuan-detection/model/目录下。

5.2　启动程序

可以使用 YuanIdea 或者 IntelliJ IDEA 运行 yuan-detection，运行效果如图 5-1 所示。

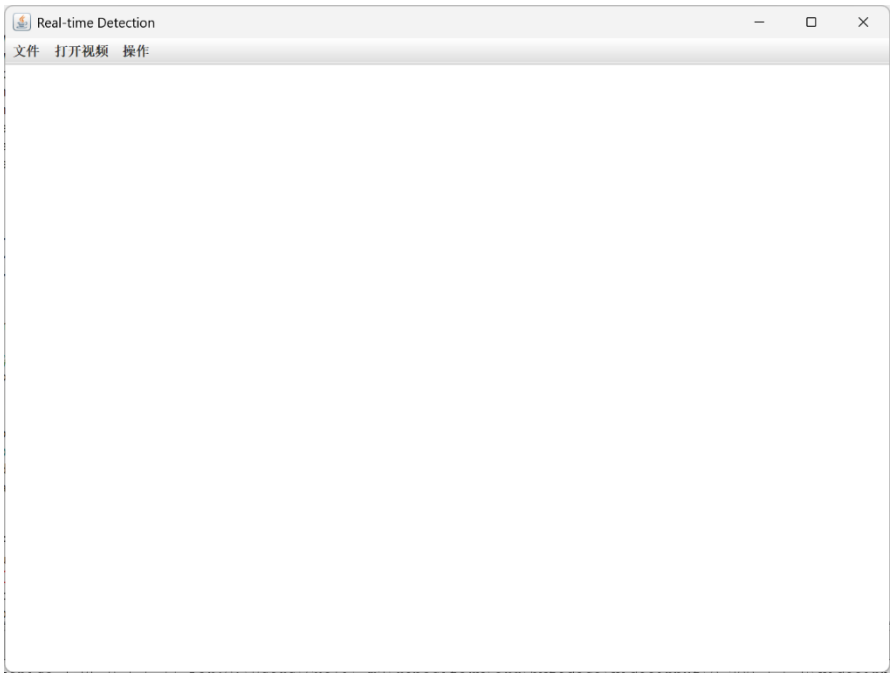


图 5-1 视频检测

5.2.1　打视频源

当前支持三种视频源，本地的 MP4，笔记本自带的摄像头，网络摄像头。

5.2.1.1　视频

选择 “打开视频” 菜单下的 “本地视频”，选择一个提前录制好的 mp4 视频，效果如图 5-2。

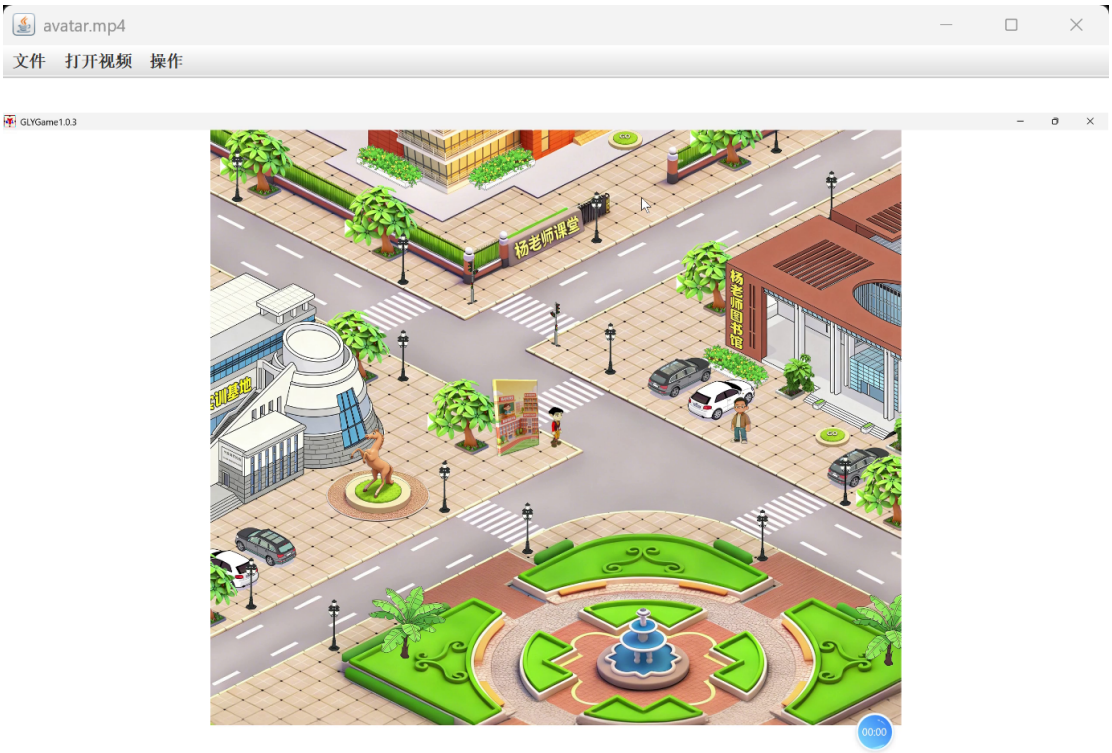


图 5-2 播放 MP4

5.2.1.2　本地摄像头

选择“打开视频”菜单下的“本摄像头”。选择“操作”菜单下的“目标追踪”，如图 5-3 为笔记本摄像头的目标追踪。

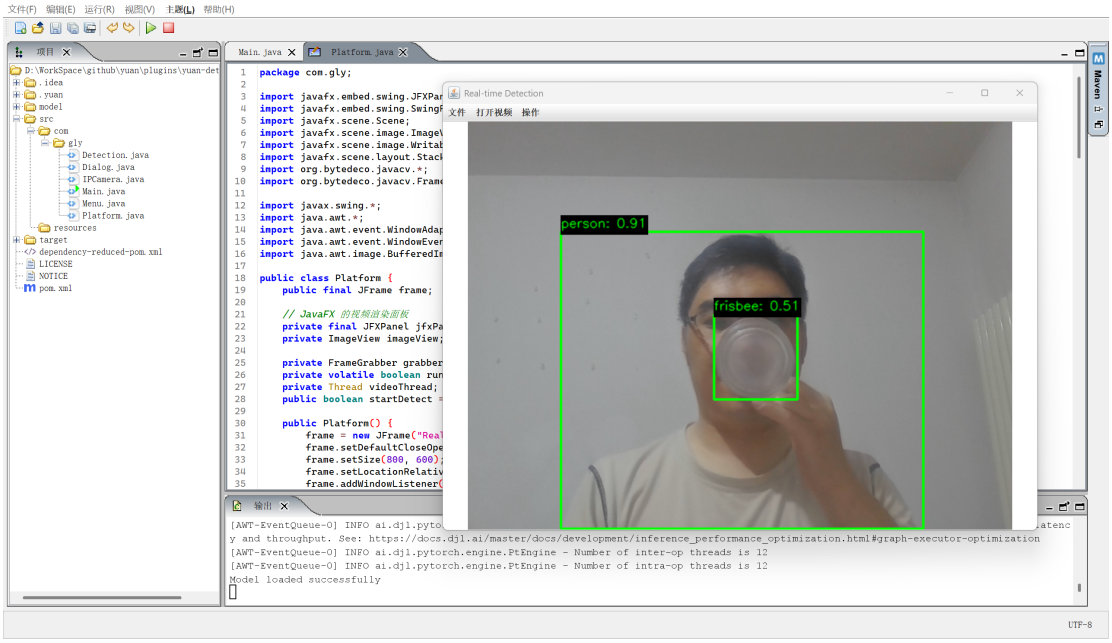


图 5-3 本地摄像头

5.2.1.3　网络摄像头

选择“打开视频”菜单下的“网络摄像头”。在图 5-4 中先择“RTSP”协议，配置网络摄像头的连接信息。



图 5-4 网络摄像头配置

选择“操作”菜单下的“目标追踪”，图 5-5 为海康网络摄像头的的目标追踪。

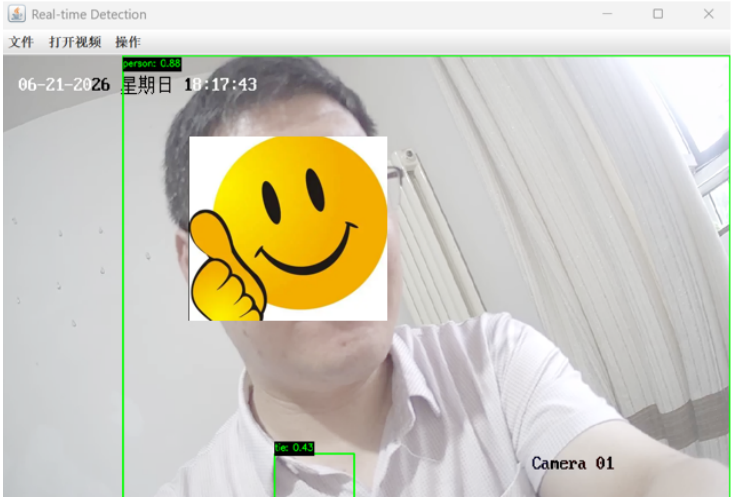


图 5-5 摄像头目标追踪

5.2.2　目标追踪

打开任意一种视频源，例如图 5-2，选择“操作”菜单下的“目标追踪”，为效果图 5-6。

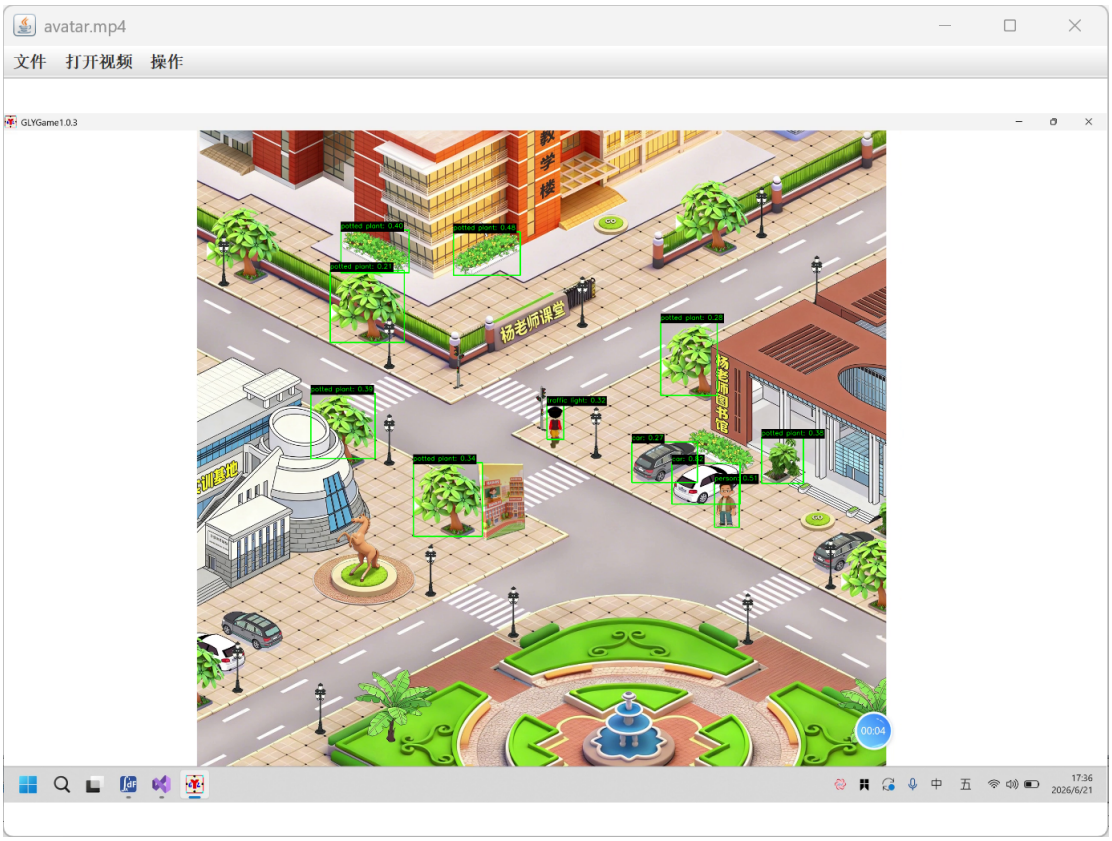


图 5-6 视频目标追踪

再次点击“目标追踪”菜单，可以取消目标追踪功能，回到普通视频状态，如图 5-2 所示。

5.2.3　切换中英文标签

点击“文件”下的“中文标签”菜单，实现中英文标签的实时切换，在显示目标追踪标签时，可以看到中英文标签效果。

5.2.4　分类截图

点击“操作”下的“保存分类截图”菜单，可以启动将追踪的分类保存到磁盘功能，保存为当前项目的 data 文件夹下。例如，将图 5-6 中的内容分类保存效果为图 5-7。

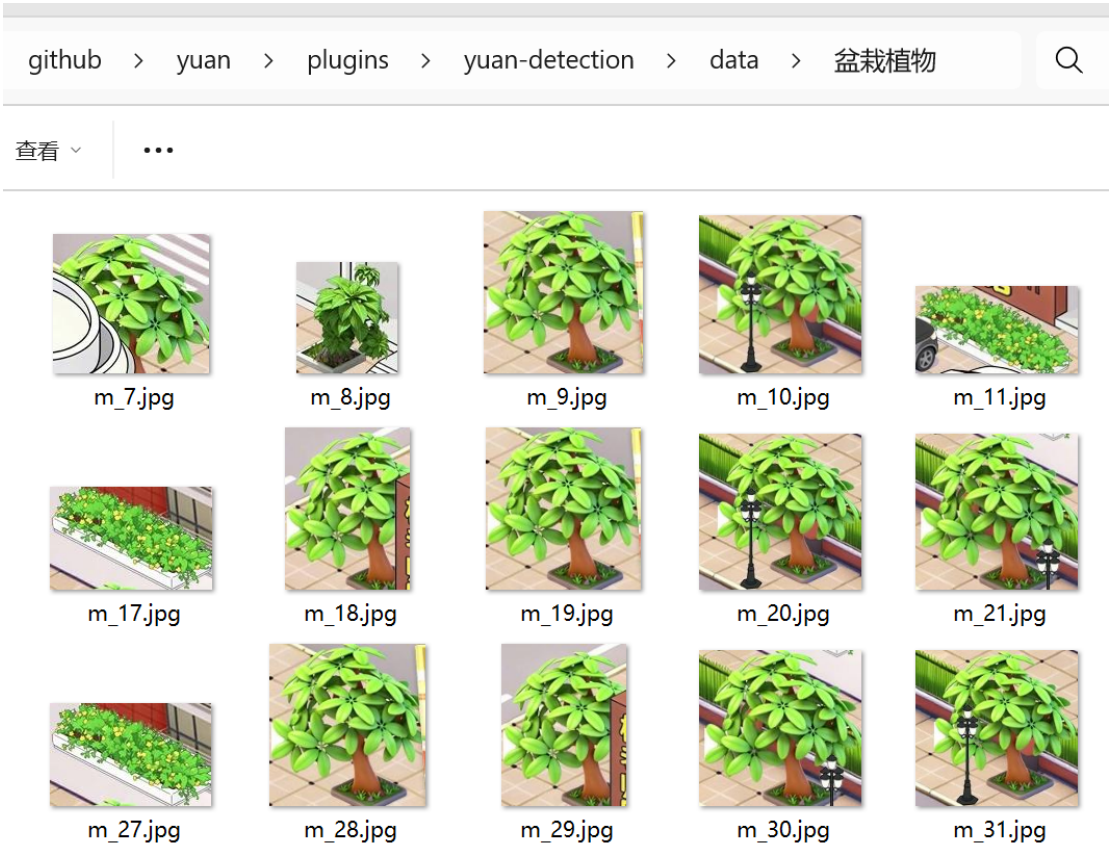

github > yuan > plugins > yuan-detection > data > 盆栽植物
查看
m_7.jpg
m_8.jpg
m_9.jpg
m_10.jpg
m_11.jpg
m_17.jpg
m_18.jpg
m_19.jpg
m_20.jpg
m_21.jpg
m_27.jpg
m_28.jpg
m_29.jpg
m_30.jpg
m_31.jpg

图 5-7 分类截图